

Universidad Nacional de Colombia

Programación Orientada a Objetos 2026-2S

Actividad 2: Valor 20%

Estudiante

Juan Diego Toro Zuluaga
C.C. 1022143298

Docente

Walter Hugo Arboleda Mazo

Repositorio GitHub

https://github.com/jtoroz/Actividad1-POO/blob/main/actividad2_poo.ipynb

8 de octubre de 2026

Ejercicio 2.1 - Clase Persona

Enunciado. Se requiere un programa que modele el concepto de una persona, con los atributos nombre, apellidos, número de documento de identidad y año de nacimiento. La clase debe tener un constructor que inicialice todos sus atributos, un método imprimir() que muestre los valores en pantalla y un método principal donde se creen dos personas y se muestren sus datos.

URL en GitHub: https://github.com/jtoroz/Actividad1-POO/blob/main/actividad2_poo.ipynb

Código fuente

```
class Persona:

    def __init__(self, nombre, apellidos, numero_documento, anio_nacimiento):
        self.nombre = nombre
        self.apellidos = apellidos
        self.numero_documento = numero_documento
        self.anio_nacimiento = anio_nacimiento

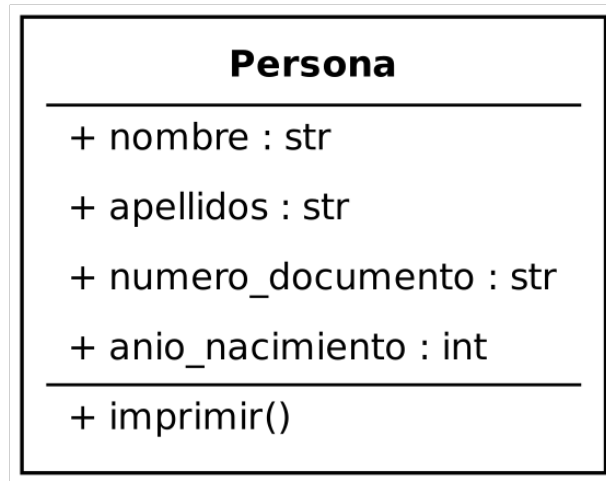
    def imprimir(self):
        print("Nombre =", self.nombre)
        print("Apellidos =", self.apellidos)
        print("Número de documento de identidad =", self.numero_documento)
        print("Año de nacimiento =", self.anio_nacimiento)
        print()

personas = []

for i in range(2):
    print("Datos de la persona", i + 1)
    nombre = input("Nombre: ")
    apellidos = input("Apellidos: ")
    documento = input("Número de documento de identidad: ")
    anio = int(input("Año de nacimiento: "))
    personas.append(Persona(nombre, apellidos, documento, anio))
    print()

for persona in personas:
    persona.imprimir()
```

Diagrama de clases



Ejercicio 2.2 - Clase Planeta

Enunciado. Se requiere un programa que modele un planeta del sistema solar con los atributos nombre, cantidad de satélites, masa, volumen, diámetro, distancia media al Sol (en millones de kilómetros), tipo de planeta (GASEOSO, TERRESTRE o ENANO) y si es observable. Debe tener constructor, método imprimir(), método calcular_densidad() que devuelva masa/volumen y método es_planeta_exterior() que determine si el planeta está más allá del cinturón de asteroides (3.4 UA, donde 1 UA = 149 597 870 km). El programa principal crea dos planetas, muestra sus atributos, su densidad y si son exteriores.

URL en GitHub: https://github.com/jtoroz/Actividad1-POO/blob/main/actividad2_poo.ipynb

Código fuente

```
from enum import Enum

class TipoPlaneta(Enum):
    GASEOSO = 1
    TERRESTRE = 2
    ENANO = 3

class Planeta:

    def __init__(self, nombre, cantidad_satelites, masa, volumen, diametro,
                 distancia_sol, tipo, es_observable):
        self.nombre = nombre
        self.cantidad_satelites = cantidad_satelites
        self.masa = masa
        self.volumen = volumen
        self.diametro = diametro
        self.distancia_sol = distancia_sol
        self.tipo = tipo
        self.es_observable = es_observable

    def imprimir(self):
        print("Nombre del planeta =", self.nombre)
        print("Cantidad de satélites =", self.cantidad_satelites)
        print("Masa del planeta =", self.masa)
        print("Volumen del planeta =", self.volumen)
        print("Diámetro del planeta =", self.diametro)
        print("Distancia al sol =", self.distancia_sol)
        print("Tipo de planeta =", self.tipo.name)
        print("Es observable =", self.es_observable)

    def calcular_densidad(self):
        return self.masa / self.volumen

    def es_planeta_exterior(self):
        # 1 UA = 149597870 km = 149.59787 millones de km
        limite = 3.4 * 149.59787
        return self.distancia_sol > limite

planetas = []

for i in range(2):
```

```

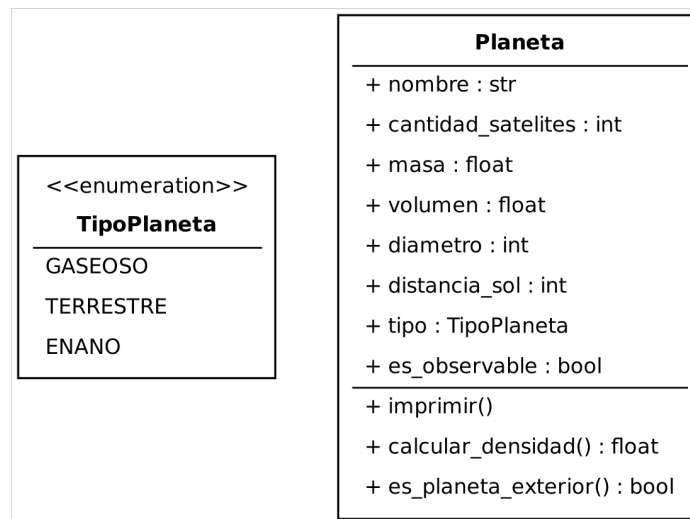
print("Datos del planeta", i + 1)
nombre = input("Nombre: ")
satelites = int(input("Cantidad de satélites: "))
masa = float(input("Masa (kg): "))
volumen = float(input("Volumen (km3): "))
diametro = int(input("Diámetro (km): "))
distancia = int(input("Distancia al Sol (millones de km): "))
tipo = TipoPlaneta[input("Tipo (GASEOSO / TERRESTRE / ENANO): ").upper()]
observable = input("¿Es observable? (s/n): ").lower() == "s"
planetas.append(Planeta(nombre, satelites, masa, volumen, diametro,
                        distancia, tipo, observable))

print()

for planeta in planetas:
    planeta.imprimir()
    print("Densidad del planeta =", planeta.calcular_densidad())
    print("Es planeta exterior =", planeta.es_planeta_exterior())
    print()

```

Diagrama de clases



Ejercicio 2.3 - Clase Automóvil

Enunciado. Se requiere un programa que modele un automóvil con los atributos marca, modelo, motor, tipo de combustible, tipo de automóvil, número de puertas, cantidad de asientos, velocidad máxima, color y velocidad actual (inicia en cero). Debe tener constructor, métodos get y set para cada atributo, los métodos acelerar(), desacelerar() y frenar(), el método calcular_tiempo_llegada() y el método imprimir(). El programa principal crea un automóvil, fija su velocidad actual en 100 km/h, acelera 20 km/h, desacelera 50 km/h y luego frena, mostrando la velocidad actual después de cada operación.

URL en GitHub: https://github.com/jtoroz/Actividad1-POO/blob/main/actividad2_poo.ipynb

Código fuente

```
from enum import Enum

class TipoCombustible(Enum):
    GASOLINA = 1
    BIOETANOL = 2
    DIESEL = 3
    BIODIESEL = 4
    GAS_NATURAL = 5

class TipoAutomovil(Enum):
    CIUDAD = 1
    SUBCOMPACTO = 2
    COMPACTO = 3
    FAMILIAR = 4
    EJECUTIVO = 5
    SUV = 6

class Color(Enum):
    BLANCO = 1
    NEGRO = 2
    ROJO = 3
    NARANJA = 4
    AMARILLO = 5
    VERDE = 6
    AZUL = 7
    VIOLETA = 8

class Automovil:

    def __init__(self, marca, modelo, motor, tipo_combustible, tipo_automovil,
                  numero_puertas, cantidad_asientos, velocidad_maxima, color):
        self.marca = marca
        self.modelo = modelo
        self.motor = motor
        self.tipo_combustible = tipo_combustible
        self.tipo_automovil = tipo_automovil
        self.numero_puertas = numero_puertas
        self.cantidad_asientos = cantidad_asientos
        self.velocidad_maxima = velocidad_maxima
        self.color = color
```

```
        self.velocidad_actual = 0

def get_marca(self):
    return self.marca

def set_marca(self, marca):
    self.marca = marca

def get_modelo(self):
    return self.modelo

def set_modelo(self, modelo):
    self.modelo = modelo

def get_motor(self):
    return self.motor

def set_motor(self, motor):
    self.motor = motor

def get_tipo_combustible(self):
    return self.tipo_combustible

def set_tipo_combustible(self, tipo_combustible):
    self.tipo_combustible = tipo_combustible

def get_tipo_automovil(self):
    return self.tipo_automovil

def set_tipo_automovil(self, tipo_automovil):
    self.tipo_automovil = tipo_automovil

def get_numero_puertas(self):
    return self.numero_puertas

def set_numero_puertas(self, numero_puertas):
    self.numero_puertas = numero_puertas

def get_cantidad_asientos(self):
    return self.cantidad_asientos

def set_cantidad_asientos(self, cantidad_asientos):
    self.cantidad_asientos = cantidad_asientos

def get_velocidad_maxima(self):
    return self.velocidad_maxima

def set_velocidad_maxima(self, velocidad_maxima):
    self.velocidad_maxima = velocidad_maxima

def get_color(self):
    return self.color

def set_color(self, color):
    self.color = color

def get_velocidad_actual(self):
    return self.velocidad_actual

def set_velocidad_actual(self, velocidad_actual):
    self.velocidad_actual = velocidad_actual

def acelerar(self, incremento):
```

```

        if self.velocidad_actual + incremento <= self.velocidad_maxima:
            self.velocidad_actual += incremento
        else:
            print("No se puede incrementar a una velocidad superior a la máxima.")

    def desacelerar(self, decremento):
        if self.velocidad_actual - decremento >= 0:
            self.velocidad_actual -= decremento
        else:
            print("No se puede decrementar a una velocidad negativa.")

    def frenar(self):
        self.velocidad_actual = 0

    def calcular_tiempo_llegada(self, distancia):
        return distancia / self.velocidad_actual

    def imprimir(self):
        print("Marca =", self.marca)
        print("Modelo =", self.modelo)
        print("Motor =", self.motor)
        print("Tipo de combustible =", self.tipo_combustible.name)
        print("Tipo de automóvil =", self.tipo_automovil.name)
        print("Número de puertas =", self.numero_puertas)
        print("Cantidad de asientos =", self.cantidad_asientos)
        print("Velocidad máxima =", self.velocidad_maxima)
        print("Color =", self.color.name)

marca = input("Marca: ")
modelo = int(input("Modelo (año): "))
motor = int(input("Motor (litros): "))
combustible = TipoCombustible[input("Combustible (GASOLINA / BIOETANOL / DIESEL / BIODIESEL / GAS_NATURAL): ")
tipo = TipoAutomovil[input("Tipo (CIUDAD / SUBCOMPACTO / COMPACTO / FAMILIAR / EJECUTIVO / SUV): ").upper()]
puertas = int(input("Número de puertas: "))
asientos = int(input("Cantidad de asientos: "))
velocidad_maxima = int(input("Velocidad máxima (km/h): "))
color = Color[input("Color (BLANCO / NEGRO / ROJO / NARANJA / AMARILLO / VERDE / AZUL / VIOLETA): ").upper()]

auto = Automovil(marca, modelo, motor, combustible, tipo, puertas, asientos,
                 velocidad_maxima, color)

print()
auto.imprimir()

auto.set_velocidad_actual(100)
print("Velocidad actual =", auto.get_velocidad_actual())

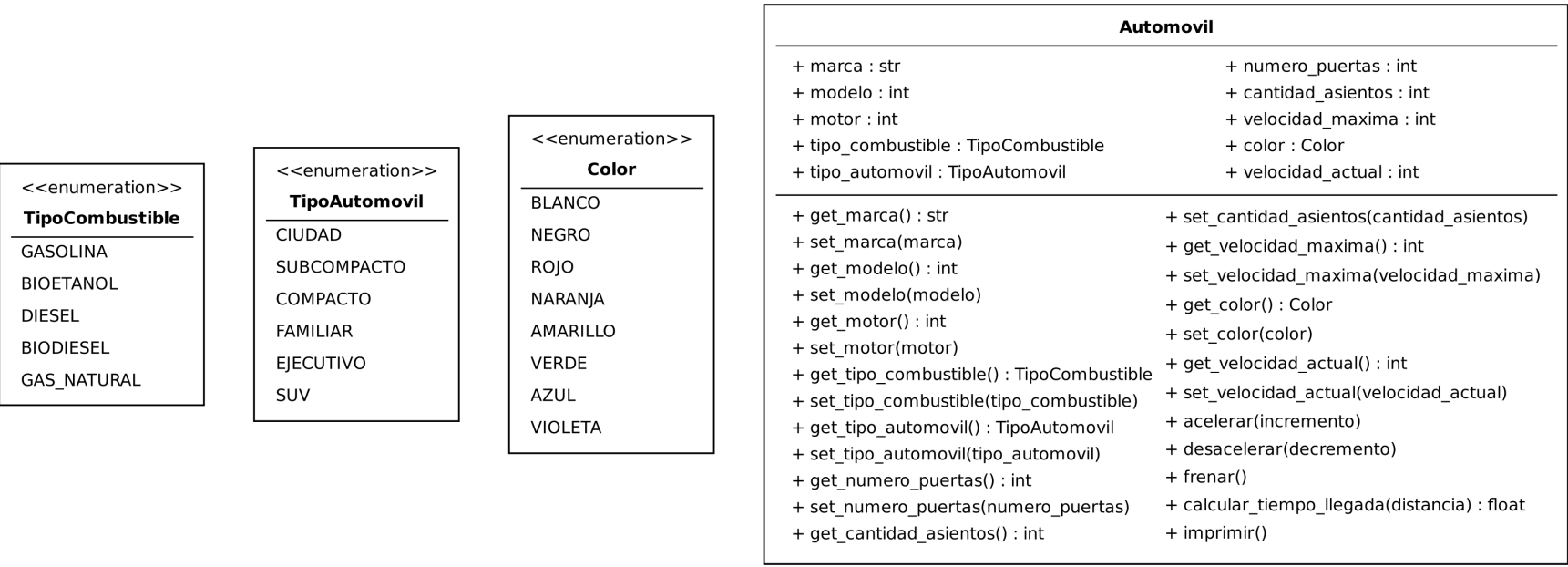
auto.acelerar(20)
print("Velocidad actual =", auto.get_velocidad_actual())

auto.desacelerar(50)
print("Velocidad actual =", auto.get_velocidad_actual())

auto.frenar()
print("Velocidad actual =", auto.get_velocidad_actual())

```

Diagrama de clases - Ejercicio 2.3 - Clase Automóvil



Ejercicio 2.4 - Figuras geométricas

Enunciado. Se requiere un programa que modele cuatro figuras geométricas: círculo (radio), rectángulo (base y altura), cuadrado (lado) y triángulo rectángulo (base y altura). Cada figura debe tener los métodos `calcular_area()` y `calcular_perimetro()`. El triángulo rectángulo además debe tener los métodos `calcular_hipotenusa()`, usando el teorema de Pitágoras, y `determinar_tipo_triangulo()`, que indique si es equilátero, isósceles o escaleno. Una clase de prueba instancia un objeto de cada figura y prueba sus métodos.

URL en GitHub: https://github.com/jtoroz/Actividad1-POO/blob/main/actividad2_poo.ipynb

Código fuente

```
import math

class Circulo:

    def __init__(self, radio):
        self.radio = radio

    def calcular_area(self):
        return math.pi * self.radio ** 2

    def calcular_perimetro(self):
        return 2 * math.pi * self.radio

class Rectangulo:

    def __init__(self, base, altura):
        self.base = base
        self.altura = altura

    def calcular_area(self):
        return self.base * self.altura

    def calcular_perimetro(self):
        return 2 * self.base + 2 * self.altura

class Cuadrado:

    def __init__(self, lado):
        self.lado = lado

    def calcular_area(self):
        return self.lado * self.lado

    def calcular_perimetro(self):
        return 4 * self.lado

class TrianguloRectangulo:

    def __init__(self, base, altura):
        self.base = base
        self.altura = altura

    def calcular_hipotenusa(self):
```

```

        return math.sqrt(self.base ** 2 + self.altura ** 2)

    def calcular_area(self):
        return self.base * self.altura / 2

    def calcular_perimetro(self):
        return self.base + self.altura + self.calcular_hipotenusa()

    def determinar_tipo_triangulo(self):
        hipotenusa = self.calcular_hipotenusa()
        if self.base == self.altura == hipotenusa:
            print("Es un triángulo equilátero")
        elif self.base != self.altura and self.base != hipotenusa and self.altura != hipotenusa:
            print("Es un triángulo escaleno")
        else:
            print("Es un triángulo isósceles")

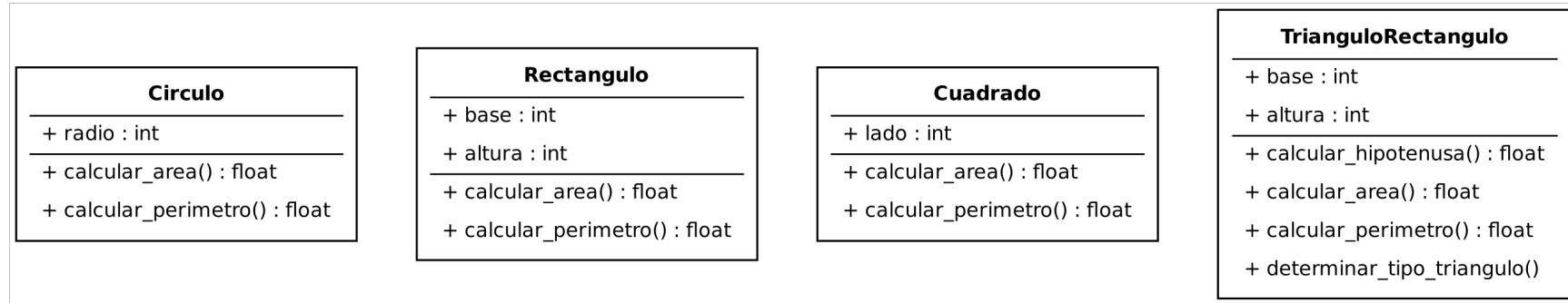
radio = int(input("Radio del círculo: "))
base_rectangulo = int(input("Base del rectángulo: "))
altura_rectangulo = int(input("Altura del rectángulo: "))
lado = int(input("Lado del cuadrado: "))
base_triangulo = int(input("Base del triángulo: "))
altura_triangulo = int(input("Altura del triángulo: "))

figura1 = Circulo(radio)
figura2 = Rectangulo(base_rectangulo, altura_rectangulo)
figura3 = Cuadrado(lado)
figura4 = TrianguloRectangulo(base_triangulo, altura_triangulo)

print()
print("El área del círculo es =", figura1.calcular_area())
print("El perímetro del círculo es =", figura1.calcular_perimetro())
print()
print("El área del rectángulo es =", figura2.calcular_area())
print("El perímetro del rectángulo es =", figura2.calcular_perimetro())
print()
print("El área del cuadrado es =", figura3.calcular_area())
print("El perímetro del cuadrado es =", figura3.calcular_perimetro())
print()
print("El área del triángulo es =", figura4.calcular_area())
print("La hipotenusa del triángulo es =", figura4.calcular_hipotenusa())
print("El perímetro del triángulo es =", figura4.calcular_perimetro())
figura4.determinar_tipo_triangulo()

```

Diagrama de clases - Ejercicio 2.4 - Figuras geométricas



Ejercicio 2.5 - Clase CuentaBancaria

Enunciado. Se requiere un programa que modele una cuenta de banco con los atributos nombres del titular, apellidos del titular, número de cuenta, tipo de cuenta (AHORROS o CORRIENTE) y saldo, que inicia en cero y por eso no se pasa al constructor. Debe tener los métodos imprimir(), consultar_saldo(), consignar(valor) (el valor debe ser mayor que cero) y retirar(valor) (el valor debe ser mayor que cero y menor o igual al saldo actual); ambos actualizan el saldo y retornan un valor booleano de confirmación. El programa principal crea una cuenta y realiza operaciones de consignación y retiro.

URL en GitHub: https://github.com/jtoroz/Actividad1-POO/blob/main/actividad2_poo.ipynb

Código fuente

```
from enum import Enum

class TipoCuenta(Enum):
    AHORROS = 1
    CORRIENTE = 2

class CuentaBancaria:

    def __init__(self, nombres_titular, apellidos_titular, numero_cuenta, tipo_cuenta):
        self.nombres_titular = nombres_titular
        self.apellidos_titular = apellidos_titular
        self.numero_cuenta = numero_cuenta
        self.tipo_cuenta = tipo_cuenta
        self.saldo = 0

    def imprimir(self):
        print("Nombres del titular =", self.nombres_titular)
        print("Apellidos del titular =", self.apellidos_titular)
        print("Número de cuenta =", self.numero_cuenta)
        print("Tipo de cuenta =", self.tipo_cuenta.name)
        print("Saldo =", self.saldo)

    def consultar_saldo(self):
        print("El saldo actual es =", self.saldo)

    def consignar(self, valor):
        if valor > 0:
            self.saldo += valor
            print("Se ha consignado $" + str(valor) + ". El nuevo saldo es $" + str(self.saldo))
            return True
        else:
            print("El valor a consignar debe ser mayor que cero.")
            return False

    def retirar(self, valor):
        if valor > 0 and valor <= self.saldo:
            self.saldo -= valor
            print("Se ha retirado $" + str(valor) + ". El nuevo saldo es $" + str(self.saldo))
            return True
        else:
            print("El valor a retirar debe ser mayor que cero y menor o igual al saldo actual.")
            return False
```

```

nombres = input("Nombres del titular: ")
apellidos = input("Apellidos del titular: ")
numero = int(input("Número de cuenta: "))
tipo = TipoCuenta[input("Tipo de cuenta (AHORROS / CORRIENTE): ").upper()]

cuenta = CuentaBancaria(nombres, apellidos, numero, tipo)

print()
cuenta.imprimir()
print()

valor_consignacion = int(input("Valor a consignar: "))
cuenta.consignar(valor_consignacion)

valor_retiro = int(input("Valor a retirar: "))
cuenta.retirar(valor_retiro)

cuenta.consultar_saldo()

```

Diagrama de clases

